

# Compte-Rendu

## Projet Noté - Calcul Parallèle

MARCHAL Enzo  
JACQUET Arthur  
SPATH Nathan  
CABOT Matthieu  
VINCENT Julien

## Introduction

Dans le cadre de ce dernier projet de programmation répartie, nous allons nous intéresser au calcul parallèle, et plus particulièrement à la parallélisation des données.

L'idée de base est assez intuitive : quand un calcul est trop lourd pour une seule machine, pourquoi ne pas le répartir sur plusieurs ? C'est exactement ce que nous allons mettre en pratique ici, à travers un exemple concret et plutôt impressionnant : le lancer de rayons (*ray tracing*). Il s'agit d'un algorithme de synthèse d'images qui simule le comportement de la lumière pixel par pixel. Le résultat est visuellement très convaincant, mais le temps de calcul devient vite problématique dès qu'on augmente la résolution.

Pour résoudre ce problème, nous avons découpé l'image en plusieurs morceaux pour distribuer chaque morceau à une machine différente. Chacune effectue sa partie du calcul, puis renvoie son résultat. Il ne reste plus qu'à tout assembler pour obtenir l'image finale. Dans ce compte rendu, nous verrons d'abord comment se comporte l'algorithme de base, puis nous concevons et implémenterons l'architecture distribuée en RMI permettant d'accélérer drastiquement le traitement.

Dans ce compte rendu, nous verrons d'abord comment se comporte l'algorithme seul, puis nous concevons et implémenterons l'architecture distribuée permettant d'accélérer le traitement.

## GIT

[https://github.com/Pugile/Projet\\_ProgReparti\\_RAIL1/tree/main](https://github.com/Pugile/Projet_ProgReparti_RAIL1/tree/main)

# Table des matières

|                                               |          |
|-----------------------------------------------|----------|
| <b>Introduction</b>                           | <b>1</b> |
| <b>GIT</b>                                    | <b>1</b> |
| <b>Table des matières</b>                     | <b>2</b> |
| <b>Répartition du travail</b>                 | <b>2</b> |
| <b>Réponses aux questions du sujet</b>        | <b>3</b> |
| Architecture imaginée                         | 3        |
| Analyse du programme fourni                   | 3        |
| Reproduction d'image                          | 4        |
| <b>Architecture complète de l'application</b> | <b>5</b> |
| Rôles des composants                          | 5        |
| Mécanisme de résilience                       | 5        |
| <b>Interfaces</b>                             | <b>6</b> |
| Interface ServiceCentral                      | 6        |
| Interface ServiceCentral                      | 6        |
| Calcule                                       | 7        |
| <b>Fonctionnement détaillé</b>                | <b>8</b> |
| Démarrage du service central                  | 8        |
| Démarrage d'un noeud de calcul                | 8        |
| Exécution côté client                         | 8        |
| Exécution côté noeud                          | 8        |
| <b>Vidéo</b>                                  | <b>9</b> |
| <b>Conclusion</b>                             | <b>9</b> |

## Répartition du travail

Service central : Enzo Marchal - Julien Vincent

Clients : Enzo Marchal - Matthieu Cabot

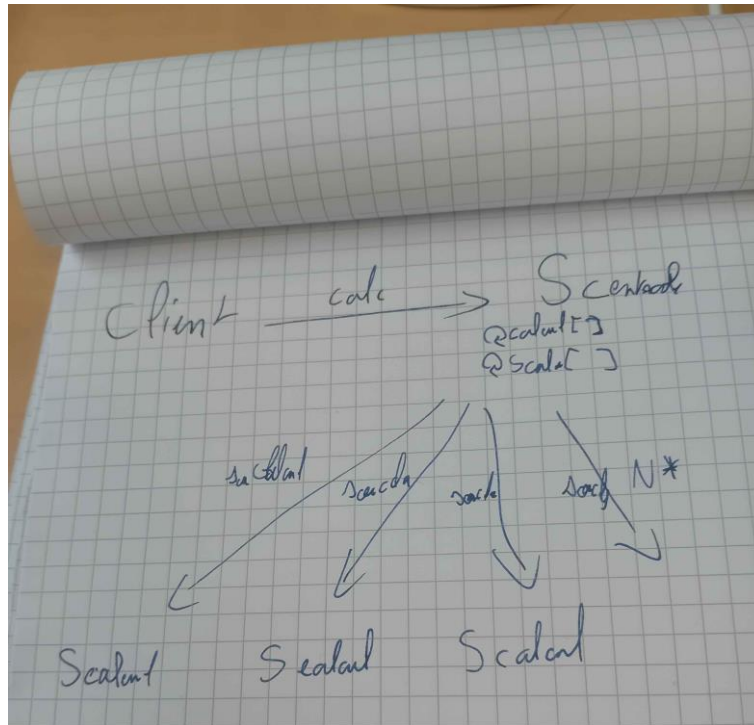
Algorithme de division de calcul : Nathan Späth - Matthieu Cabot

Service calcul : Arthur Jacquet

Compte rendu / Vidéo : Matthieu Cabot - Nathan Späth

# Réponses aux questions du sujet

## Architecture imaginée



Pour répondre à cette question, nous avons imaginé l'architecture suivante. Le client envoie sa demande de calcul à un serveur de nœuds (Scentral), qui joue le rôle d'intermédiaire. Ce serveur maintient deux listes : une première contenant les adresses des nœuds de calcul disponibles (Scalcul[]), et une seconde contenant les morceaux de calcul à effectuer (calcul[]). Quand une demande arrive, le serveur associe chaque morceau de calcul à un nœud disponible, puis ouvre une connexion socket vers chacun d'eux pour leur transmettre leur tâche. Le nombre de nœuds est variable (noté  $N^*$  sur le schéma), ce qui rend l'architecture flexible. Puisque chaque nœud (Scalcul) traite sa portion de manière totalement indépendante, les calculs se font en parallèle, ce qui permet de réduire drastiquement le temps de rendu de l'image finale.

Après réflexion et discussion avec notre professeur, nous avons constaté que cette première approche n'était pas optimale. Nous avons migré vers une architecture plus adaptée, décrite dans la section 4.

## Analyse du programme fourni

Un premier algorithme de création et d'affichage d'image via Raytracing nous a été fourni, qui nous permettait de visualiser la génération d'une image suivant sa taille. L'analyse de l'algorithme Scene.compute montre que le temps de calcul est proportionnel au nombre de

pixels, et donc croît de manière quadratique avec la résolution. Par exemple, si l'on double la largeur et la hauteur (passant de 512x512 à 1024x1024), le nombre de pixels est multiplié par 4, tout comme le temps de rendu global. Cela démontre l'absolue nécessité d'adopter une approche distribuée pour des images en haute définition.

## Reproduction d'image

En ne modifiant que LancerRaytracer.java, il est possible d'appeler `scene.compute()` plusieurs fois avec des paramètres de décalage différents, puis d'afficher chaque résultat dans la même fenêtre via `disp.setImage()` :

```
int l = largeur/2, h = hauteur/2;
// Quart haut-gauche
Image img1 = scene.compute( x0: 0, y0: 0, l, h);
disp.setImage(img1, x0: 0, y0: 0);
// Quart bas-droite
Image img2 = scene.compute(l, h, l, h);
disp.setImage(img2, l, h);
```

Cette modification illustre concrètement la capacité du moteur de rendu à calculer des portions indépendantes de la scène, posant ainsi les bases de la parallélisation.

# Architecture complète de l'application

## Rôles des composants

- Service Central (ServiceCentralImpl) :

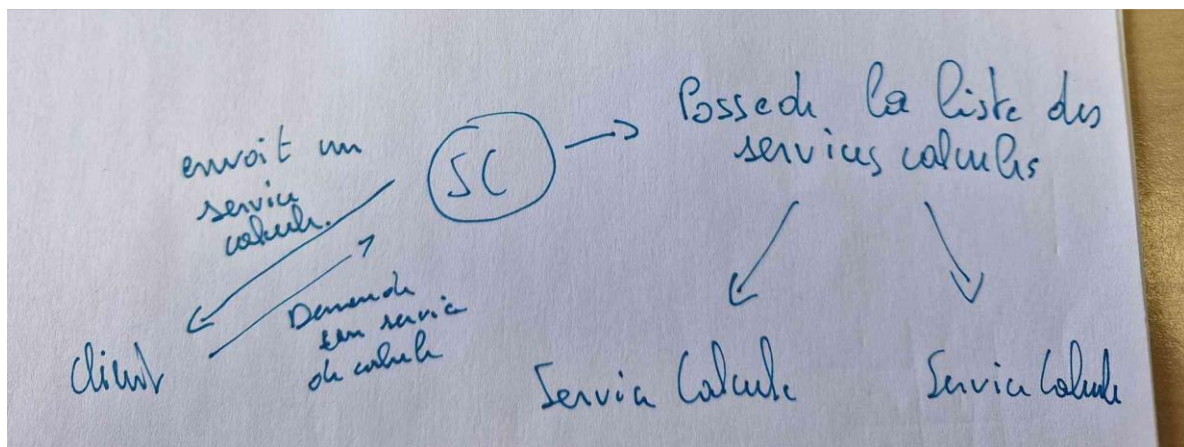
Processus fixe, Tient à jour la liste des nœuds actifs, distribue les références de nœuds aux clients et retire les nœuds défaillants sur signalement.

- Nœud de calcul (ServiceCalculeImpl) :

Processus mobile. S'enregistre auprès du Service Central, puis attend des tâches RMI et exécute le raytracing sur la portion reçue et retourne l'image partielle.

- Client (Clients)

Processus Mobile. Découpe l'image en un certain nombre de bandes. Lance 1 thread par bande en parallèle, chacun obtenant un nœud via le Service Central, lui envoyant son calcul et affichant le résultat.



## Mécanisme de résilience

Si un nœud de calcul ne répond pas lors d'un appel RMI, le client intercepte la `RemoteException` et appelle `updateService(noeud)` sur le Service Central, qui retire immédiatement ce nœud de sa liste. La bande concernée n'est pas recalculée dans l'implémentation actuelle (voir section 9 pour les pistes d'amélioration).

# Interfaces

## Interface ServiceCentral

Cette interface définit les méthodes exposées par le service central via RMI. Elle hérite de Remote, obligatoire pour Java RMI. Chaque méthode déclare throws RemoteException pour gérer les erreurs réseau.

```
public interface ServiceCentral extends Remote { 10 usages 1 implementation  🧑 Arthur JAcquet +1
    ServiceCalcule envoyerService() throws RemoteException; 1 usage 1 implementation  🧑 Arthur JAcquet
    void enregisterClient(ServiceCalcule noeud) throws RemoteException; 1 usage 1 implementation  🧑 Arthur JAcquet
    void updateService(ServiceCalcule noeud) throws RemoteException; 1 usage 1 implementation  🧑 Matthieu Cabot
}
```

| Méthode                 | Description                                                      |
|-------------------------|------------------------------------------------------------------|
| envoyerService()        | Retourne un stub RMI vers un nœud disponible.                    |
| enregisterClient(noeud) | Enregistre un nœud de calcul dans la liste des nœuds actifs.     |
| updateService(noeud)    | Retire un nœud défaillant de la liste sur signalement du client. |

## Interface ServiceCentral

Cette interface définit le contrat de chaque nœud de calcul. Elle ne comporte qu'une seule méthode : recevoir un objet Calcule et retourner un objet Image, tous deux sérialisés via RMI.

```
public interface ServiceCalcule extends Remote { 1 implementation  🧑 Nathan Spath
    Image calcule(Calcule calcule) throws RemoteException; 1 implementation  🧑 Nathan Spath
}
```

| Méthode            | Description                                                                                                                  |
|--------------------|------------------------------------------------------------------------------------------------------------------------------|
| calcule(Calcule c) | Reçoit la description de la portion à calculer, exécute l'algorithme de raytracing et retourne l'image partielle sérialisée. |

## Calcule

La classe Calcule est l'objet de transfert entre le client et les nœuds de calcul. Elle implémente Serializable pour être transmise via RMI. Elle encapsule toutes les informations nécessaires à un nœud pour effectuer son calcul de manière complètement autonome :

| Champ            | Type   | Rôle                                                           |
|------------------|--------|----------------------------------------------------------------|
| sceneDescription | String | Chemin vers le fichier de description de la scène (simple.txt) |
| startX           | int    | Coordonnée X de départ de la portion                           |
| startY           | int    | Coordonnée Y de départ                                         |
| blockWidth       | int    | Largeur de la bande à calculer par ce nœud                     |
| blockHeight      | int    | Hauteur de la bande à calculer par ce nœud                     |
| totalWidth       | int    | Largeur totale de l'image                                      |
| totalHeight      | int    | Hauteur totale de l'image                                      |

# Fonctionnement détaillé

## Démarrage du service central

Le Service Central est le premier processus à démarrer. Il crée un objet `ServiceCentralImpl`, l'exporte via `UnicastRemoteObject` pour qu'il soit accessible à distance, puis le lie dans le RMI Registry sous le nom "ServiceCentral" sur le port 1099. À partir de ce moment, il est prêt à recevoir des connexions.

`ServiceCentralImpl` maintient une `ArrayList<ServiceCalcule>` de toutes les références RMI des nœuds enregistrés. La sélection du prochain nœud à utiliser se fait en round-robin via un compteur entier `i`, réinitialisé à 0 quand il dépasse la taille de la liste.

## Démarrage d'un noeud de calcul

Chaque nœud peut être démarré sur une machine distincte du réseau. Son adresse IP, son port dans l'annuaire et le port du Service Central lui sont passés en arguments. Au démarrage, il :

1. Créer un objet `ServiceCalculeImpl` et l'exporte via `UnicastRemoteObject` sur le port demandé.
2. Se connecte au RMI Registry du Service Central.
3. Appelle `enregistrerClient(stub)` sur le Service Central pour s'y inscrire.
4. Attend ensuite indéfiniment les appels RMI entrants.
5. Réalise les calculs reçues un par un et envoie le résultat.

## Exécution côté client

Le client se connecte au Service Central via le RMI Registry. L'utilisateur saisit la largeur, la hauteur et le nombre de coupes souhaitées. La méthode `diviserCalcule()` découpe l'image par le nombre de coupes demandées en bandes horizontales de hauteur égale (le reste est ajouté à la dernière bande). Pour chaque bande, un thread Java indépendant est créé. Chaque thread :

1. Appelle `envoyerService()` pour obtenir un stub vers un nœud disponible.
2. Construit un objet `Calcule` avec les coordonnées de sa bande.
3. Appelle `calcule(c)` sur le nœud — le raytracing s'exécute à distance.
4. Reçoit un objet `Image` sérialisé en retour.
5. Appelle `disp.setImage()` pour afficher sa bande dans la fenêtre graphique.

Les threads s'exécutant simultanément, le calcul est réellement distribué et parallèle sur l'ensemble des nœuds disponibles.

## Exécution côté noeud

Quand un nœud reçoit un appel `calcule(Calcule c)`, il recrée une instance de `Scene` à partir du nom de fichier et des dimensions totales, puis appelle `scene.compute(startX,`



startY, width, height) avec les coordonnées de la portion à calculer. L'objet Image résultant est retourné et automatiquement sérialisé par RMI pour être transmis au client.

## Vidéo

La vidéo de présentation a été réalisée sur un ordinateur sous linux Ubuntu de la salle 125. Sur la vidéo on voit 2 nœuds de calcul travaillant sur le "localhost", un autre nœud travaille en parallèle sur l'ordinateur placé à côté de lui qui est enregistré sur le même point central.

2 Images de taille 1000 x 1000 sont réalisées avec une découpe de 1000 coupes.

## Conclusion

Ce projet nous a permis de mettre en pratique de manière concrète les concepts fondamentaux de la programmation répartie et du calcul parallèle. En distribuant le calcul d'une image raytracing sur plusieurs nœuds reliés en réseau, nous avons pu observer les gains en performance apportés par la parallélisation.